Yandex Cloud

Облачные технологии в работе с большими данными. День 10.

Баташов Руслан

Elasticsearch

Высокомасштабируемая распределенная поисковая система полнотекстового поиска и анализа данных с веб-интерфейсом, REST API и неформализованными JSON-документами.

Широко применяется для аналитической обработки больших массивов данных, анализа логов и журналов работы приложений, функций автозавершения и умного ввода текста.

Elasticsearch

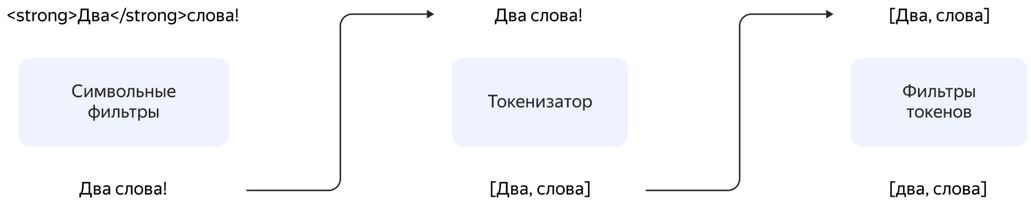
Сейчас Elasticsearch — ядро экосистемы Elastic Stack, в состав которой также входят:

- механизм сбора данных и регистрации журналов Logstash;
- платформа аналитики и визуализации Kibana;
- платформа для одноцелевых отправителей данных Beats.

Elasticsearch. Поиск

Elasticsearch — это поиск в режиме реального времени, горизонтально масштабируемый поиск и поддержка многопоточности. Основное преимущество системы — именно в качестве и скорости обработки текста, в том числе при полнотекстовом поиске по всем выражениям во всех документах вашей базы данных.

Основой для работы с текстовыми документами является анализатор. Он представляет собой цепочку последовательных обработчиков. Сначала поступивший в анализатор текст проходит символьные фильтры, которые убирают, добавляют или заменяют отдельные символы в потоке.



Затем Elasticsearch передает обработанный текст токенизатору, который очищает поток символов от знаков препинания и разбивает по определенным правилам на отдельные слова — токены.

После токенизатора система передает набор слов в один или несколько фильтров токенов, которые могут добавлять, удалять или менять слова в наборе. На выходе из анализатора мы имеем набор токенов, который помещается в индекс. Это позволяет сохранять максимум смысла при минимуме знаков.

Elasticsearch ищет слова из запроса уже по индексу. При этом поисковые индексы можно разделить на сегменты — шарды. На каждом узле (запущенном экземпляре Elasticsearch) может быть размещено несколько сегментов. Каждый узел действует как координатор для делегирования операций правильному сегменту, а перебалансировка и маршрутизация выполняются автоматически.

Elasticsearch. Хранилище данных

Elasticsearch — это нереляционное хранилище документов, которое имеет REST API и позволяет работать с данными в формате JSON. Строгая структурированность при этом не является обязательным требованием.

Основные преимущества Elasticsearch — это высокая горизонтальная масштабируемость, легкое управление и отказоустойчивость. Система может быть запущена на десятках или сотнях узлов вместо одного мощного сервера, а добавить в кластер еще один узел можно практически без дополнительных настроек.

Elasticsearch. Использование

- **Для полнотекстового поиска по сайту**

Stack Overflow использует Elasticsearch как средство для полнотекстового поиска по вопросам и ответам для пользователей, а также для поиска похожих вопросов и подсказок при создании нового вопроса. С помощью Elasticsearch сервис предоставляет поиск по точному совпадению (например, поиск строки кода) и нечёткий поиск с большим количеством настроек.
GitHub обеспечивает пользователей возможностями полнотекстового поиска и поиска по отдельным критериям среди 8 миллионов репозиториев кода благодаря Elasticsearch. Например, можно найти проект на языке Clojure, который был активен в течение последнего месяца.
Альфа-Банк применяет Elasticsearch для полнотекстового поиска по транзакциям в личном кабинете и в сервисе «Выписка по счёту»

- **Для хранения и анализа журналов**

Netflix выбрал Elasticsearch для хранения, индексации и поиска по этим данным. Всего за два года после начала внедрения инфраструктура выросла до 150 кластеров, а количество инстансов превысило 3500.
ДомКлик организовал кластер логирования с помощью Elasticsearch. Задача кластера — агрегировать и вести поиск по журналам более 2000 приложений, которые работают в кластере Kubernetes. Система обрабатывает более 4,4 терабайтов данных в сутки.

- **Для поиска по продуктам**

Компания Leroy Merlin в России с помощью Elasticsearch создала поиск по продуктам, быструю выдачу каталога и сервис поисковых подсказок.
С момента запуска сервиса количество товаров в индексе увеличилось в два раза и продолжает расти. В среднем пользователи делают 300 поисковых запросов в секунду, а время отклика составляет меньше 200 миллисекунд.

- **Для визуализации и анализа показателей**

Тинькофф предоставляет мониторинг как сервис на основе Elasticsearch более чем 200 командам компании. Каждую секунду на него поступает один гигабайт данных от балансировщиков, гипервизоров и коммутаторов, информация о состоянии биржи или платежей и данные о пользовательских инцидентах.

Redis

Высокопроизводительная СУБД класса NoSQL с открытым исходным кодом, работающая с данными типа «ключ-значение».

Redis.

В key-value хранилище хранятся пары ключей и значений. Ключ в паре — это идентификатор, а значение — любые данные: число, строка или сложный объект.

В Redis вместо SQL в запросах используются скрипты на языке Lua. В отличие от других подобных хранилищ, Redis поддерживает больше типов данных (строки, списки, хеш-таблицы, упорядоченные и неупорядоченные множества, битовые массивы и др.) и операций с ними.

Redis хранит данные в оперативной памяти и благодаря этому обеспечивает очень высокую производительность — миллионы операций в секунду. Это же порождает два принципиальных ограничения:

- Объём хранимых данных не может превышать размера оперативной памяти.
- При сбое сервера всё, что ещё не сохранилось на диск, безвозвратно пропадёт. Так что не стоит использовать Redis в качестве основного хранилища данных.

Redis. Использование

- Создавать сервер очередей (механизм «публикация-подписка»).
- Кешировать результаты запросов к основной базе данных.
- Кешировать промежуточные результаты при обработке данных.
- Управлять сессиями пользователей и счетчиками: количество пользователей онлайн, просмотры и т.д.

kafka

Kafka

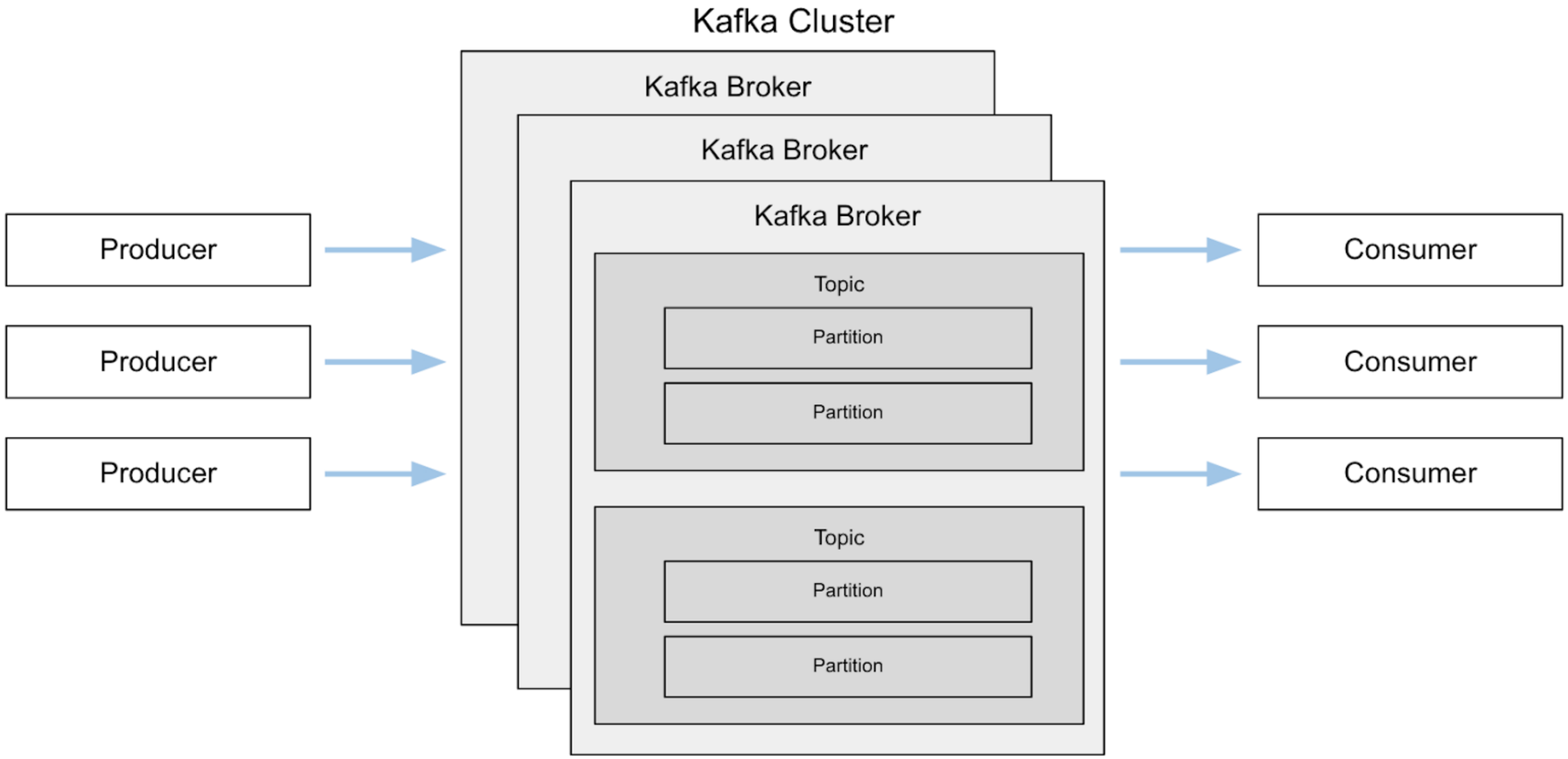
Отказоустойчивая распределённая стриминговая платформа с открытым исходным кодом, которая позволяет хранить, обрабатывать и доставлять огромные объёмы данных в реальном времени.

Kafka

Kafka — гибрид распределённой базы данных и брокера сообщений с возможностью горизонтального масштабирования. Kafka собирает у приложений данные, хранит их в своем распределённом хранилище, группируя по топикам, и отдаёт компонентам приложения по подписке. При этом сообщения хранятся на различных узлах-брокерах, что обеспечивает высокую доступность и отказоустойчивость.

Топик — это способ группировки потоков сообщений в хранилище по категориям. Сервисы публикуют сообщения определённой категории в топик, а потребители подписываются на топик и читают из него сообщения. Для каждого топика Apache Kafka ведёт лог сообщений, который может быть разбит на несколько разделов. **Разделы** — это последовательность сообщений топика в порядке поступления.
Сообщения сохраняются в так называемом **журнале** — долговременной упорядоченной структуре данных. Записи в журнал можно только добавлять, их нельзя ни изменять, ни удалять, а информация считывается слева направо, что гарантирует правильный порядок элементов.

Kafka


Kafka Cluster
Kafka Broker
Kafka Broker
Kafka Broker
Topic
Partition
Partition
Topic
Partition
Partition
Producer
Producer
Producer
Consumer
Consumer
Consumer

Kafka. Использование

- **Масштабные IoT/IIoT-системы**, характеризующиеся архитектурой с множеством датчиков, сенсоров, контроллеров и других конечных устройств.
- **Системы аналитики**. Например, Kafka используется в компаниях IBM и DataSift в качестве коллектора для мониторинга событий и трекера потребления потоков данных пользователями в режиме реального времени.
- **Финансовые системы**. Используют финансовые организации: Сбербанк, Тинькофф, Альфа-Банк, ING Bank.
- **Социальные сети.** В Twitter Kafka — часть инфраструктуры потоковой обработки, а в LinkedIn используется для потоковой передачи данных о деятельности и операционных показателях приложений.
- **Системы геопозиционирования.** Foursquare — для передачи сообщений между онлайн- и офлайн-системами, а также для интеграции средств мониторинга в свою big data инфраструктуру на базе Hadoop.
- **Телеком-операторы**. ВымпелКом, МТС, Ростелеком и др.
- **Онлайн-игры.** Например, Demonware, подразделение Activision Blizzard — для обработки логов пользователей.
- Самый простой пример: с помощью Apache Kafka можно собирать логи сеансов от клиентов в потоковом режиме или логи с физических файлов журналов с серверов, а затем помещать их в одном месте, например в HDFS — файловой системе Apache Hadoop. Также сервис позволяет построить конвейер данных, чтобы с помощью алгоритмов машинного обучения извлекать из сырой информации сведения, ценные для бизнеса

Хранение и анализ данных

1. Объектное хранилище. Создание бакетов и загрузка объектов.
2. БД. Создание кластера БД и подключение к ней. PostgreSQL/MySQL/MongoDB/ClickHouse
3. Hadoop. Создание и подключение к кластеру, работа с Hive.
4. Создание датасетов, чартов и дашбордов в DataLens.

Параметры больших данных

- **Volume** — объём — от 150 Гб в сутки.
- **Velocity** — скорость. Объём и содержимое Big Data ежесекундно меняются, поэтому собирать и обрабатывать их нужно на больших вычислительных мощностях. Например, сервис FlightRadar24, где отображаются все маршруты самолётов в режиме онлайн.
- **Variety** — разнообразие. Массив больших данных может включать фото, видео и тексты, файлы разных объёмов и форматов, данные из множества разных источников. Обычные данные, как правило, однородные, например таблица Excel с Ф. И. О. каждого сотрудника.
- **Veracity** — достоверность. Большие данные собирают только из источников, которым можно доверять, а для анализа используют точные и объективные методы. Поэтому корпорации и международные организации принимают стратегические решения на основе этих данных.
- **Variability** — изменчивость. Большие данные обновляются в режиме онлайн, поэтому их поток нестабилен. На него влияют скорость передачи, изменение источников, действия пользователей и даже смена сезонов. При анализе данных нужно учитывать и прогнозировать все эти факторы. Например, данные об авиаперелётах стоит использовать с поправками на задержки рейсов и погодные условия, из-за которых меняются маршруты.
- **Value** — ценность. Сами по себе данные ничего не значат, но на их основе можно сделать глубокие выводы и принимать взвешенные решения. Например, проанализировать трафик на дорогах в течение года и понять, как лучше построить маршруты для городского транспорта.

Параметры больших данных

- Структурированными — то есть уже размеченными по определённым параметрам. Допустим, данные о ключевых медицинских показателях пациентов: температура, давление, анализы крови и ЭКГ.

- Частично структурированными, например файлы разного формата с записями о стихийных бедствиях в регионе за последние пять лет.

- Неструктурированными, например фото, музыка и сообщения всех пользователей VK.

  Ещё одно важное отличие больших данных от обычных — распределённая структура. Это значит, что управлять ими и анализировать их можно с помощью множества микросервисов.

Как работает технология Big Data

1. Сбор
Большие данные собирают из разных источников:

- **Социальные** — всё, что публикуют и делают пользователи в соцсетях, онлайн-сервисах и приложениях. Сюда относят фото, видео, аудио, сообщения в мессенджерах, геолокации и хештеги.

- **Статистические** — все данные от госорганов и исследовательских компаний о людях, животных, транспортных средствах, товарах и услугах, политических и экономических явлениях.

- **Медицинские** — данные из электронных карт о медицинских показаниях, анализах, аппаратной диагностике, вакцинациях, историях болезней.

- **Машинные** — записи с камер наблюдения, видеорегистраторов, систем управления и умных устройств.

- **Транзакционные** — данные о платежах и переводах через банки и другие финансовые сервисы.

В процессе сбора данные проходят очистку, или Data Cleaning. На этом этапе, с помощью специальных программ, данные находят, отбирают и фильтруют, проверяя на точность и соответствие заданным параметрам. Специалисты по Data Cleaning размечают массивы данных так, чтобы алгоритмам было проще находить нужные сегменты информации в ответ на запросы пользователей.

Данные извлекают полностью или частично — с момента последнего успешного извлечения. Например, когда составляют прогноз погоды, используют только данные за последние сутки. Чтобы извлечь Big Data за определённый период, в хранилище используют специальную функцию захвата данных.

Как работает технология Big Data

2. Хранение

Для хранения больших данных используют:

- **DWH** — или data warehouse — единое хранилище для всех данных, на основе которых компания формирует отчёты и принимает решения. Файлы в них сгруппированы по областям применения и расположены по хронологии. Например, DWH интернет-магазина, где собраны данные обо всех клиентах, транзакциях и подразделениях. Данные в них поступают по принципу ETL (от англ. Extract, Transform, Load): сначала извлекаются, затем трансформируются, а потом загружаются в едином формате.

- **Data Lake** — озёра данных, которые не имеют единого формата и чёткой структуры. Порядок действий здесь такой: извлечение, загрузка в базу и трансформация в формат, который подходит для текущих задач. Озеро данных напоминает виртуальный диск, где хранятся тексты, фото и PDF, а база данных — это таблица, где все они перечислены.

- **СУБД** — системы управления базами данных, бывают реляционными или нереляционными. Для работы с большими данными чаще используются первые — данные в них организованы в виде таблиц, которые связаны между собой ключами, а для запросов используют специальный язык — SQL. Например, в колоночной СУБД ClickHouse, которую используют в сервисе Яндекс Метрика, все данные о трафике сайтов хранятся в нескольких таблицах. Строки — это события, например просмотры страниц, а колонки — параметры, например переходы с мобильных устройств. Это позволяет за секунды сформировать отчёт о трафике для сайта по нескольким параметрам сразу, поскольку все они расположены в соседних ячейках.

Как работает технология Big Data

3. Обработка

Информацию большого объёма с помощью обычных инструментов обработать будет сложно: на это уйдёт слишком много времени. Для этих задач применяют особое ПО, которое работает по технологии MapReduce. Сначала алгоритм отбирает данные по заданным параметрам, затем распределяет между отдельными узлами, серверами или компьютерами, а потом они одновременно обрабатывают эти сегменты данных, параллельно друг с другом.

Вот примеры сервисов, которые используют MapReduce:

- Hadoop — сервис с открытым исходным кодом, позволяющий собирать, хранить и работать с Big Data сразу нескольким специалистам. Он автоматически перераспределяет нагрузку так, чтобы при отказе одного из узлов другие продолжили работать вместо него.

- Apache Spark — сервис из нескольких библиотек для работы с потоковыми данными, которые обновляются с высокой скоростью. При этом данные внутри можно фильтровать, обрабатывать и применять для машинного обучения нейросетей.

  Чтобы следить за качеством сбора данных, структурировать их и находить нужное, некоторые компании нанимают специалистов — DWH-аналитиков.

Как работает технология Big Data

4. Анализ

Чтобы применять большие данные в работе, необходимо анализировать их по самым разным параметрам. В этом помогают:

- SQL — язык запросов, который применяют при работе с реляционными СУБД.
- Нейросети, натренированные с помощью машинного обучения так, чтобы за секунды обработать тонны информации и представить точные данные для самых сложных задач.

  Чтобы извлекать нужные сегменты информации и преобразовывать их в понятные отчёты и графики, используют специальные аналитические сервисы на базе Business Intelligence (BI).

Преимущества технологии Big Data

Большие данные — драйвер мировой экономики. Они помогают:

- Работать с большими объёмами информации.
  Например, базы данных миллионов пользователей VK: у каждого из них сотни сообщений, фотографий, музыки и видео.

- Строить более точные прогнозы и принимать более взвешенные решения.
  Например, планировать рекламную кампанию, опираясь на информацию Яндекс Метрики о миллионах пользователей и цифровом следе каждого из них — браузер, настройки, посещаемые сайты. Это позволит, предположим, показывать рекламу запчастей только тем, кто посещает автомобильные сайты.

- Мгновенно реагировать на сбои и уязвимости.
  Благодаря доступу к большим данным обо всех действиях пользователей, банки или платёжные сервисы могут сразу отследить подозрительные действия и остановить мошенников.

- Строить долгосрочные стратегии.
  Если у компании есть данные о продажах, прибыли и убытках за несколько лет, их анализ поможет планировать инвестиции, работу с персоналом и ассортиментом.

- Исправлять ошибки и улучшать продукт.
  Предположим, специалисты службы доставки заметили, что в вечернее время люди часто отменяют доставку через 30 минут после заказа. Это значит, что клиенты не хотят долго ждать и предпочитают сходить в магазин сами. Проблему можно решить, увеличив число курьеров в эти часы.

Минусы

- **Трудности с масштабированием.**
  К примеру, сервисом стартапа пользуются 10 тыс. человек. После рекламной кампании приходит 1 млн новых пользователей. Не каждая система для хранения и обработки данных справится с таким резким притоком. Решить проблему помогают специальные облачные хранилища, которые можно масштабировать в любой момент.

- **Высокие риски.**
  Большие данные повышают требования к безопасности. Например, если взломают базу данных крупных банков, миллионы клиентов лишатся денег. Чтобы этого избежать, компании-владельцы Big Data используют распределённый доступ: у разных групп сотрудников разный уровень доступа и только к определённым сегментам баз данных. Кроме того, данные шифруют и структурируют на каждом уровне.

- **Высокие затраты.**
  Большие данные требуют больших вычислительных мощностей, более дорогих сервисов для хранения и обработки. Допустим, для обучения нейросетей нужны огромные датасеты, которые есть только у очень крупных корпораций и часто недоступны для свободного пользования. Для работы с большими данными нужно привлекать специалистов: аналитиков данных, DWH-аналитиков, специалистов по BI.

Hadoop

Как понять, что данные по-настоящему большие? Обычно к таким относят данные, размер которых увеличивается хотя бы на 100-150 ГБ в день. Данные при этом могут быть и структурированными, и нет.

Чтобы работать с такими объёмами данных в реальном времени, необходимы специальные технологии. В их основе лежат три принципа:

1. **Распределённое хранение и обработка.** Данных много, а их обработка требует больших вычислительных ресурсов, поэтому необходимы распределённые системы из большого количества узлов. Создать такую систему и обеспечить её высокую производительность — непростая задача.

2. **Горизонтальная масштабируемость.** Данные постоянно накапливаются, а задачи их обработки варьируются от относительно простых до очень сложных. Поэтому нужна возможность легко наращивать объёмы хранилища и гибко изменять количество узлов, на которых обрабатываются данные.

3. **Отказоустойчивость.** Узлов в кластере много (сотни или даже тысячи), и вероятность того, что они будут выходить из строя, довольно высока. Следует построить распределённую систему так, чтобы сбой на одном или нескольких узлах не влиял на её работу в целом.

BIG DATA

ИНФРАСТРУКТУРНЫЕ ПРОФЕССИИ

Сотрудники облачных провайдеров, которые обеспечивают технический сбор и хранение данных, дата-инженеры и разработчики ЦОДов — центров обработки данных

АНАЛИТИЧЕСКИЕ ПРОФЕССИИ

Системные, веб- и аналитики данных, маркетологи

Их задача — обработать большие данные, чтобы сделать сервис более удобным для пользователей. Благодаря Big Data рекомендательные сервисы YouTube так хорошо подстраиваются под каждого пользователя и советуют ему подходящее видео

СПЕЦИАЛИСТЫ ПО ИИ И МАШИННОМУ ОБУЧЕНИЮ

Используют Big Data, чтобы обучать нейросети и создавать роботизированные сервисы на их основе

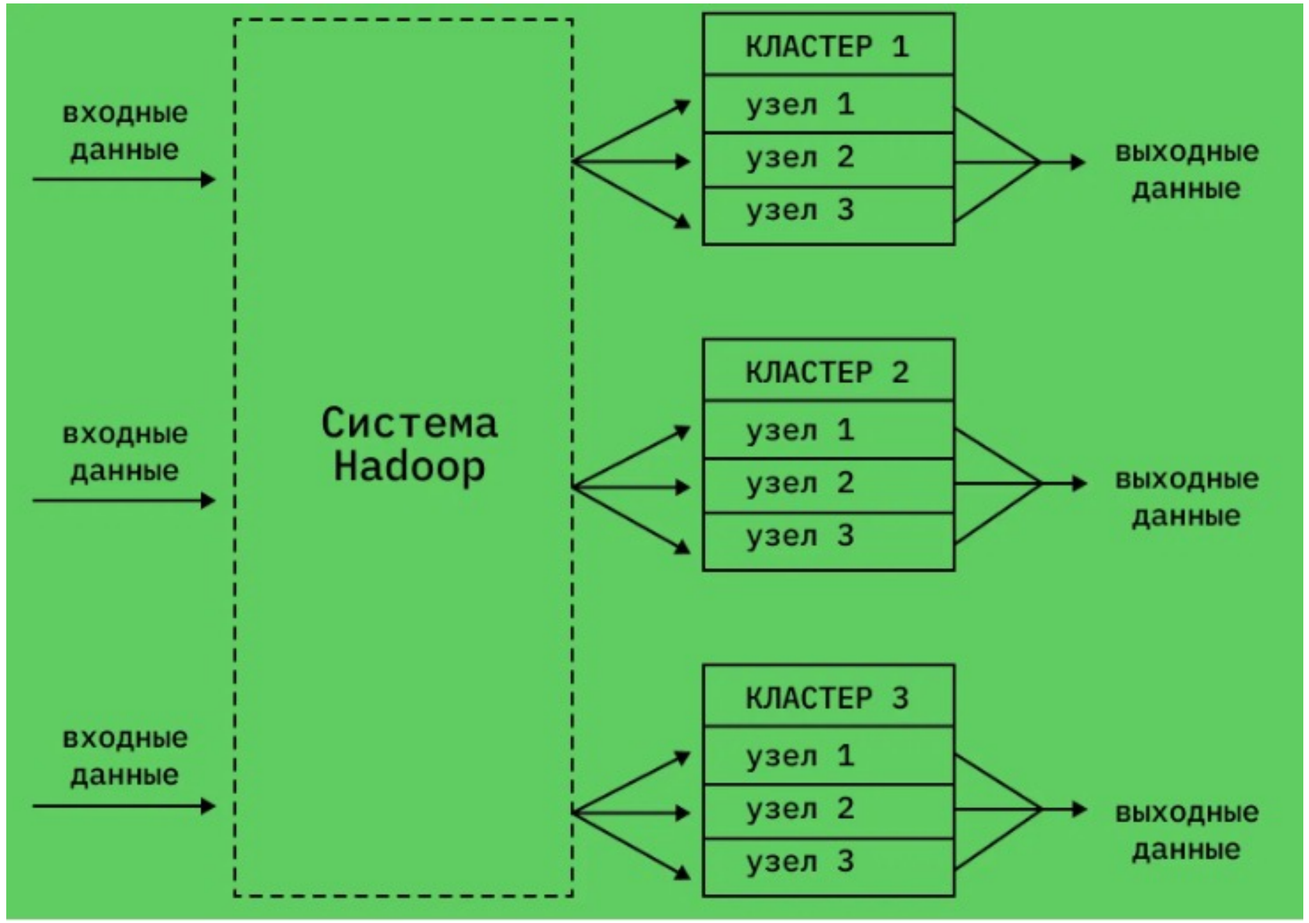
Hadoop

Платформа с открытым исходным кодом для надежной, масштабируемой, распределенной обработки больших наборов данных с помощью простых моделей программирования.

Набор утилит и библиотек для работы с данными, распределёнными между несколькими кластерами, каждый из которых состоит из сотен или даже тысяч узлов.

Hadoop

Hadoop работает по принципу MapReduce, то есть распределения данных. Его суть в том, что система состоит из кластеров — групп отдельных подсерверов, или узлов, которые используются как единый ресурс. Когда на кластер поступает обширная задача, Hadoop делит её на много мелких подзадач и выполняет каждую на своём узле. Это позволяет параллельно решать несколько задач и быстрее выдать конечный результат.



Архитектура экосистемы Hadoop

Внутри проекта Hadoop — четыре основных модуля:

1. **Hadoop Common**.
Набор инструментов, которые используются для создания инфраструктуры и работы с файлами. По сути, это управляющая система для остальных модулей и связи с дополнительными инструментами.

2. **HDFS (Hadoop Distributed File System)**.
Распределённая файловая система для хранения данных на различных узлах. В неё встроена система дублирования данных, чтобы обеспечить их надёжность и сохранность даже при отказе отдельных серверов. Здесь хранятся неструктурированные данные, извлекать которые нужно не просто запросами, а специальными средствами и инструментами.

3. **YARN (Yet Another Resource Negotiator)**.
Система управления кластером Hadoop, которая позволяет приложениям использовать вычислительные мощности.

4. **Hadoop MapReduce**.
Платформа, которая отвечает за MapReduce-вычисления, то есть распределяет входные данные по узлам.

Архитектура экосистемы Hadoop

Также в системе Hadoop есть многочисленные дополнительные компоненты, например:

● **Hive**.
Хранилище, которое позволяет запрашивать из HDFS большие наборы данных и создавать сложные задания MapReduce. Использует язык запросов HQL, который напоминает SQL. Именно с этим хранилищем, как правило, работают аналитики данных.

● **Pig**.
Инструмент для преобразования данных, который умеет подготавливать данные различных форматов для будущего использования.

● **Flume**.
Инструмент для приёма больших данных и отправки их в HDFS. Умеет собирать большие потоковые данные, например, из логов.

● **Zookeeper**.
Координатор, который помогает эффективнее распределять информацию по разным узлам.

Функции технологии

Hadoop нужен, чтобы:

● Повысить скорость обработки данных благодаря модели MapReduce и параллельным вычислениям.

● Обеспечить устойчивость данных за счёт хранения резервных копий на других узлах.

● Работать с данными любых типов и видов, в том числе неструктурированными, например видео.

Благодаря этим возможностям Hadoop используют в следующих практических задачах:

● Создание озера данных — недорогого и удобного хранилища для неструктурированной информации.

● Обработка данных из соцсетей — они часто не структурированы, поступают в больших объёмах, но позволяют получить как можно больше информации о клиентах.

● Анализ поведения клиентов, сбор информации об их действиях на сайте и в магазине, который нужен, чтобы делать на основе этого прогнозы.

● Управление рисками, анализ логов, реагирование на поломки и нарушения безопасности. Логи, как правило, идут большим потоком, и другие хранилища с ними не справляются.

● Обработка данных от интернета вещей — датчиков, размещённых на различных устройствах.

Apache Spark

Все операции в MapReduce подразумевают чтение с жёсткого диска и запись на него. Часто время, необходимое для этих операций, в разы превышает время самих вычислений. Поэтому технология MapReduce хорошо работает при распределённых вычислениях в пакетном режиме: когда данные подаются на обработку отдельными большими пакетами. А вот для обработки потоков данных в реальном времени она не подходит. Для таких задач разработаны фреймворки распределённой потоковой обработки данных, наиболее популярный из них — это **Apache Spark.**

Одно из основных отличий Spark от Hadoop заключается в том, что он хранит результаты промежуточных вычислений в памяти, не записывая их на диск. Это даёт большой прирост в производительности.

В основе Spark лежит движок, управляющий планированием и оптимизацией выполнения заданий, а также работой с источником данных (HDFS, объектным хранилищем или БД). Поверх ядра работают библиотеки. Вот основные из них:

- Spark SQL — чтобы запускать SQL-подобные команды в распределённых наборах данных;
- MLlib — для машинного обучения;
- Structured Streaming — для потоковой обработки данных в реальном времени;
- GraphX — для задач с графами.

Spark и Hadoop используются для разных задач и дополняют друг друга.

Spark & Hadoop

SPARK

- нужно анализировать потоковые данные в реальном времени;
- скорость решения задачи принципиально важна;
- задачи включают много параллельных операций с использованием итерационных алгоритмов;
- задачи связаны с машинным обучением

HADOOP

- обрабатывать большие наборы данных, когда объём даже промежуточных результатов вычислений превышает доступную память;
- создавать инфраструктуру анализа данных при ограниченном бюджете (Spark требует много оперативной памяти, так что его использование более затратно);
- решать задачи, если времени на них достаточно и не требуется немедленно получить результаты;
- заниматься пакетной обработкой данных, выполнять много операций чтения с диска и записи на него (например анализ исторических и архивных данных).

Хранение и анализ данных

1. Объектное хранилище. Создание бакетов и загрузка объектов.
2. БД. Создание кластера БД и подключение к ней. PostgreSQL/MySQL/MongoDB/ClickHouse
3. Hadoop. Создание и подключение к кластеру, работа с Hive.
4. Создание датасетов, чартов и дашбордов в DataLens.

Yandex Data Proc

Создавать кластеры Hadoop и Spark вручную — непростое и небыстрое занятие. Нужно настроить виртуальные машины (ВМ), развернуть сервисы, изменить множество конфигурационных файлов...
Сервис Yandex Data Proc автоматически создаст кластеры Hadoop или Spark, настроит сеть, установит ПО и обновит его, когда выйдет новая версия. В сервисе есть интерфейсы запуска заданий и инструменты мониторинга. Data Proc интегрируется с другими сервисами Yandex Cloud и автоматически масштабирует ресурсы.

Компоненты Data Proc

- **Tez** — фреймворк для обработки больших данных, содержащий ряд улучшений технологии MapReduce.
- **Spark** — фреймворк для распределённой потоковой обработки данных.
- **Hive** — платформа для хранения больших данных в распределённом хранилище и для управления ими.
- **ZooKeeper** — служба, координирующая работу приложений. Она хранит информацию о настройках системы, обеспечивает синхронизацию распределённого выполнения групповых задач, выявляет конфликтующие задачи и нерациональное использование ресурсов.
- **HBase** — распределённая NoSQL база данных (БД), основанная на модели Google BigTable и использующая HDFS. Её основная задача — хранить очень большие таблицы (миллиарды строк и миллионы столбцов) на узлах кластера.

Yandex Data Proc

- **Sqoop** — инструмент для передачи данных между Hadoop и реляционными БД. С его помощью можно импортировать данные из реляционных СУБД в Hadoop, преобразовать их с использованием MapReduce, а затем экспортировать обратно.
- **Oozie** — инструмент для управления рабочим процессом и координации заданий MapReduce. Может объединить несколько задач в единое логическое задание.
- **Flume** — распределённая служба, которая собирает, сортирует и перемещает большие объёмы данных журнала событий. Она может обрабатывать потоковые данные, позволяя создавать аналитические приложения для всей экосистемы Hadoop.
- **Livy** — служба, которая через REST-интерфейс обеспечивает взаимодействие с кластером Spark, включая отправку заданий или частей кода Spark, а также синхронное или асинхронное получение результатов.
- **Zeppelin** — многопользовательский инструмент для анализа и визуализации данных в браузере, а также совместной работы над данными с использованием Spark. Позволяет создавать запросы к данным в Hadoop на SQL, Scala или Python и отображать результаты в виде таблиц, графиков и диаграмм.

Сервис Yandex Data Proc интегрирован с Yandex DataSphere — облачной средой для разработки моделей машинного обучения. С её помощью можно пройти весь цикл создания моделей: от эксперимента и разработки до запуска готовой версии на вычислительных мощностях Yandex Cloud. DataSphere использует бессерверные вычисления, но если вам предстоит решать ресурсоёмкие задачи, то расчёты можно запустить и на вашем кластере Data Proc.

Yandex Data Proc

Кластер Data Proc

Основная сущность сервиса Data Proc — **кластер**. Он объединяет все ресурсы, доступные Hadoop: вычислительные мощности и хранилище.
Каждый кластер состоит из **подкластеров**. Подкластеры объединяют хосты, выполняющие идентичные функции:

- подкластер с управляющими хостами (например NameNode для HDFS или ResourceManager для YARN);
- подкластер для хранения данных (например DataNode для HDFS);
- подкластеры для обработки данных (например NodeManager для YARN).

Подкластеры каждого кластера должны находиться в одной облачной сети и одной зоне доступности.

Хранение данных
Чтобы вы не переплачивали за хранение большого объёма данных на вычислительных узлах, в сервисе реализована связь с объектным хранилищем. В него помещается основной объём данных, а на вычислительных узлах хранятся только горячие данные, к которым нужен быстрый доступ.
Вы можете запускать задания по SSH — без непосредственного доступа к кластеру Data Proc. Поэтому, чтобы вам было удобно, журнал выполнения заданий находится в отдельном бакете в объектном хранилище. Записи в журнал делаются от имени сервисного аккаунта, указанного при создании кластера.

Сеть
Все подкластеры должны находиться в одной сети, а все хосты каждого подкластера — в определённой подсети этой сети.
У хостов кластера нет публичного IP-адреса. Чтобы подключиться к кластеру Data Proc, используйте ВМ, расположенную в той же облачной сети, что и кластер. То есть вы создадите ВМ, к которой подключитесь по SSH, а с этой машины, в свою очередь, подключитесь к кластеру.

Практическая работа. Создание кластера Hadoop

На этом уроке вы создадите и настроите кластер Hadoop с помощью сервиса Yandex Data Proc. Hadoop предназначается для работы с большими данными, поэтому создание кластера потребует от вас больше усилий, чем на предыдущих практических работах (но гораздо меньше, чем если бы вы делали это самостоятельно).

Создание кластера

1. Создайте произвольный бакет в объектном хранилище для хранения зависимостей заданий нашего кластера и результатов их выполнения
2. Создайте сервисный аккаунт для доступа к кластеру. Обратите внимание: можно использовать только аккаунт с ролью mdb.dataproc.agent. Для автоматического масштабирования кластера сервисному аккаунту также понадобятся роли editor и dataproc.agent.
3. Откройте каталог, где будете создавать кластер, и выберите сервис **Data Proc**. В открывшемся окне нажмите кнопку **Создать кластер**.

Задайте для кластера имя и выберите **версию образа — 2.0** В образ включена одна из версий Hadoop и дополнительные компоненты. Некоторые вы можете устанавливать по выбору. Кроме того, в каждую версию образа входит Conda (менеджер окружений для Python) и набор инструментов машинного обучения (scikit-learn, TensorFlow, CatBoost, LightGBM и XGBoost). Обратите внимание на то, что некоторые из сервисов обязательны, чтобы использовать другие. На следующем уроке нам понадобится сервис HIVE. Выберите его, и рядом с MAPREDUCE и YARN вы увидите напоминания о том, что они нужны для HIVE.

4. Вставьте в поле **публичный ключ** публичную часть SSH-ключа.
5. Выберите созданный **сервисный аккаунт** для доступа к кластеру.
6. Выберите **зону доступности** для кластера. Все подкластеры будут находиться в этой зоне.
7. Если нужно, задайте **свойства** Hadoop и его компонентов. Доступные свойства перечислены в документации.
8. Выберите **бакет** в объектном хранилище, где будут храниться зависимости заданий и результаты их выполнения.
9. Выберите или создайте **сеть** для кластера. Включите опцию NAT в интернет для подсетей, в которых размещается кластер. Инструкция здесь.
10. Включите опцию **UI Proxy**, чтобы получить доступ к веб-интерфейсам компонентов Data Proc. У некоторых компонентов (например Hadoop, Spark, YARN и Zeppelin) есть пользовательские веб-интерфейсы, доступные на мастер-узле кластера. С помощью этих интерфейсов вы можете:
    - отслеживать ресурсы кластера и управлять ими (YARN Resource Manager, HDFS NameNode);
    - просматривать статус и отлаживать задания (Spark History, JobHistory);
    - проводить эксперименты, совместно работать или выполнять отдельные операции (Zeppelin).

Подробности об интерфейсах вы найдёте в документации.

Настройка подкластеров

В состав кластера входит один главный подкластер (**Мастер**) с управляющим хостом, а также подкластеры для хранения данных (**Data**) или вычислений (**Compute**).
В подкластерах Data можно разворачивать компоненты для хранения данных, а в подкластерах Compute — для обработки данных. Хранилище в подкластере Compute предназначено только для временного хранения обрабатываемых файлов.
Для каждого подкластера можно задать число и класс хостов, размер и тип хранилища, а также подсеть той сети, в которой расположен кластер. Кроме того, для подкластеров Compute можно настроить автоматическое масштабирование. Это позволит выполнять задания на обработку данных быстрее без дополнительных усилий с вашей стороны. Создадим подкластер Compute с одним хостом.

11. В блоке **Добавить подкластер** нажмите кнопку **Добавить**.
12. В поле **Роли** выберите COMPUTENODE. В блоке **Масштабирование** включите опцию Автоматическое масштабирование.

Автоматическое масштабирование подкластеров обработки данных поддерживается в кластерах Yandex Data Proc версии 1.2 и выше. Чтобы оно работало, в кластере с установленным Spark или Hive должен быть также установлен сервис YARN.
Yandex Data Proc автоматически масштабирует кластер, используя для этого системные метрики нагрузки на кластер. Когда их значение выходит из установленного диапазона, запускается масштабирование. Если значение метрики превысит порог, в подкластер добавятся хосты. Если опустится ниже порога, начнётся декомиссия (высвобождение ненужных ресурсов), а избыточные хосты удалятся.
По умолчанию для масштабирования используется внутренняя метрика YARN (yarn.cluster.containersPending). Она показывает, сколько единиц ресурсов нужно заданиям в очереди. Выбирайте эту опцию **Масштабирование по умолчанию**, если в кластере выполняется много относительно небольших заданий.
Другой вариант — масштабирование на основе метрики загрузки процессора (vCPU). Чтобы использовать его, отключите опцию **Масштабирование по умолчанию** и укажите целевой уровень загрузки vCPU.

Подключение к кластеру и работа с Hive

На этом уроке вы научитесь подключаться к кластеру Hadoop и работать с ним на примере выполнения запросов с помощью Hive.

1. Выберите машину, которую создавали раньше, или создайте новую. Подключитесь к ней по SSH.
2. Подключитесь с этой машины к хосту главного подкластера также с помощью SSH. Для этого на машине должна быть закрытая часть SSH-ключа, открытую часть которого вы указали при создании кластера Data Proc. Вы можете скопировать ключ на виртуальную машину или подключаться к ней с запущенным SSH-агентом.

```
sudo nano ~/.ssh/<имя ключа>
```

3. В открывшийся редактор скопируйте содержимое закрытой части SSH-ключа с вашей локальной машины.
4. Запустите SSH-агент:

```
eval `ssh-agent -s`
```

5. Добавьте ключ в список доступных агенту:

```
ssh-add ~/.ssh/<имя ключа>
```

6. Узнайте внутренний FQDN хоста главного подкластера. Для этого в консоли управления на странице кластера перейдите на вкладку Хосты и выберите хост с ролью MASTERNODE.

```
ssh root@<FQDN хоста>
```

Пошаговые инструкции по различным способам подключения к кластеру Data Proc приведены в документации.

Вы должны получить примерно такой результат:

```
admin@vm1:~$ ssh-add /home/admin/.ssh/rsa
Enter passphrase for /home/admin/.ssh/rsa:
Identity added: /home/admin/.ssh/rsa (ipivnev@ipivnev-osx)
admin@vm1:~$
admin@vm1:~$
admin@vm1:~$ ssh root@rc1b-dataproc-m-2gyn7a8qreypfgpr.mdb.yandexcloud.net
The authenticity of host 'rc1b-dataproc-m-2gyn7a8qreypfgpr.mdb.yandexcloud.net (10.129.0.24)' can't be established.
ECDSA key fingerprint is SHA256:1M+XlCzTFuMLJrKc3fQKveBaXTYVFfEYN1WLk8uA5xU.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'rc1b-dataproc-m-2gyn7a8qreypfgpr.mdb.yandexcloud.net,10.129.0.24' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 16.04.7 LTS (GNU/Linux 4.4.0-142-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
root@rc1b-dataproc-m-2gyn7a8qreypfgpr:~#
root@rc1b-dataproc-m-2gyn7a8qreypfgpr:~#
root@rc1b-dataproc-m-2gyn7a8qreypfgpr:~#
```

7. Проверим, что команды Hadoop выполняются, например:hadoop version

```
root@rc1b-dataproc-m-2gyn7a8qreypfgpr:~# hadoop version
Hadoop 2.10.0
Subversion https://github.yandex-team.ru/mdb/bigtop.git -r c2a1b44e8e6d6ebeca495dd77cc7025b9ae98a82
Compiled by dataproc on 2020-09-11T12:40Z
Compiled with protoc 2.5.0
From source with checksum 7b2d8877c5ce8c9a2cca5c7e81aa4026
This command was run using /usr/lib/hadoop/hadoop-common-2.10.0.jar
root@rc1b-dataproc-m-2gyn7a8qreypfgpr:~#
```

Запуск заданий Apache Hive

Hive — это платформа для хранения данных и управления ими в экосистеме Hadoop. Она используется для доступа к большим датасетам, сохранённым в распределённом хранилище.
Hive позволяет работать с данными различного формата (csv, tsv, Parquet, ORC, Avro и другими), подключаться к БД и взаимодействовать с ней с помощью SQL-подобного языка запросов. Hive используется преимущественно для работы с данными в HDFS, HBase, S3-совместимых хранилищах и реляционных СУБД.

Запрос на действия с данными в Hive называется **заданием**. Задания можно запускать на управляющем хосте с помощью командной оболочки CLI Hive, а также с помощью CLI Yandex Cloud.

1. Для запуска Hive CLI выполните команду hive на управляющем хосте.
2. Проверьте, всё ли работает: выполните, например, команду select 1; — корректный результат выглядит так:

```
ubuntu@rc1b-dataproc-m-zazadao4rv1w2k32:~$ hive
/usr/lib/hadoop/libexec/hadoop-functions.sh: line 2366: HADOOP_ORG.APACHE.HADOOP.HBASE.UTIL.GETJAVAPROPERTY_USER: invalid variable name
/usr/lib/hadoop/libexec/hadoop-functions.sh: line 2461: HADOOP_ORG.APACHE.HADOOP.HBASE.UTIL.GETJAVAPROPERTY_OPTS: invalid variable name
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/lib/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/hadoop/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = 7fd82d76-9dce-4333-bb58-3ecf6d0492c1

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j2.properties Async: true
Hive Session ID = 5ceb40b2-61a9-4277-b24e-0c8abcd8a2fc
2023-04-17 21:37:34,437 INFO  [pool-7-thread-1] client.RMProxy: Connecting to ResourceManager at rc1b-dataproc-m-zazadao4rv1w2k32.mdb.yandexcloud.net/10.129.0.31:8032
2023-04-17 21:37:34,437 INFO  [Tez session start thread] client.RMProxy: Connecting to ResourceManager at rc1b-dataproc-m-zazadao4rv1w2k32.mdb.yandexcloud.net/10.129.0.31:8032
hive> 2023-04-17 21:37:35,027 INFO  [Tez session start thread] client.AHSProxy: Connecting to Application History server at rc1b-dataproc-m-zazadao4rv1w2k32.mdb.yandexcloud.net/10.129.0.31:1020
2023-04-17 21:37:35,030 INFO  [pool-7-thread-1] client.AHSProxy: Connecting to Application History server at rc1b-dataproc-m-zazadao4rv1w2k32.mdb.yandexcloud.net/10.129.0.31:10200

    >
    >
    > select 1;
OK
1
Time taken: 3.54 seconds, Fetched: 1 row(s)
```

3. Теперь создайте внешнюю таблицу (external table) в формате Parquet, содержащую открытые данные о списке перелётов между городами США в 2018 году. Для этого с помощью Hive CLI выполните запрос:

CREATE EXTERNAL TABLE flights (Year bigint, Month bigint, FlightDate string, Flight_Number_Reporting_Airline bigint, OriginAirportID bigint, DestAirportID bigint) STORED AS PARQUET LOCATION 's3a://yc-mdb-examples/dataproc/example01/set01’;

4. Проверим список таблиц, выполнив команду show tables. Результат должен выглядеть так:

```
hive>
    >
    > show tables;
OK
flights
Time taken: 0.041 seconds, Fetched: 1 row(s)
```

5. Запросим число перелётов с разбивкой по месяцам:

SELECT Month, COUNT(*) FROM flights GROUP BY Month;

```
hive> SELECT Month, COUNT(*) FROM flights GROUP BY Month;
Query ID = ubuntu_20230417213953_0650ca63-5cc6-489e-92dd-5e5f4599a05e
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
2023-04-17 21:39:54,804 INFO  [7fd82d76-9dce-4333-bb58-3ecf6d0492c1 main] client.RMProxy: Connecting to ResourceManager at rc1b-dataproc-m-zazadao4rv1w2k32
.mdb.yandexcloud.net/10.129.0.31:8032
2023-04-17 21:39:54,804 INFO  [7fd82d76-9dce-4333-bb58-3ecf6d0492c1 main] client.AHSProxy: Connecting to Application History server at rc1b-dataproc-m-zaza
dao4rv1w2k32.mdb.yandexcloud.net/10.129.0.31:10200
Session re-established.
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1681767116644_0006)

--------------------------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      6          6        0        0       0       0
Reducer 2 ...... container     SUCCEEDED     10         10        0        0       0       0
--------------------------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 18.16 s
--------------------------------------------------------------------------------------------------
OK
5       616529
8       644673
9       585749
6       626193
7       645299
10      616101
11      586178
1       570118
3       611987
4       596046
12      593842
2       520731
Time taken: 25.612 seconds, Fetched: 12 row(s)
hive>
```

Если вас интересует работа с большими данными в облаке, посмотрите доклады сотрудников Yandex Cloud об управлении кластерами Hadoop и заданиями в Data Proc на YouTube-канале Yandex Cloud.